Informe del Trabajo Práctico Integrador

Sistema de Gestión de Inventario

Materia: Fundamentos de Informática · Comisión Viernes Noche · 2.º cuatrimestre 2026

Integrantes: Facundo Rodriguez y Diego Fernandez Escobar

Versión documentada: proyecto original publicado en GitHub, commit f8d4e3d (rama main).

1. Objetivo y estructura de datos

El programa permite administrar el inventario de un pequeño comercio desde un menú de consola. Incluye altas, bajas, modificación de stock, búsquedas por código, ordenamientos y reportes. Como ampliaciones opcionales, cuenta con una interfaz gráfica desarrollada con tkinter y exportación a JSON.

Los productos se almacenan en una matriz (lista de listas): cada fila representa un producto y cada columna conserva una posición fija. El archivo inventario.csv contiene una fila de encabezado y 15 productos de prueba.

Columna	Dato	Tipo / ejemplo
0	Código único	Entero · 101
1	Nombre	Texto · Yerba 1kg
2	Categoría	Texto · Almacén
3	Precio unitario	Número · 3500.0
4	Stock actual	Entero · 12

2. Funciones principales

Grupo	Funciones y finalidad
Persistencia	cargar_csv(nombre_archivo): lee el CSV y arma la matriz. guardar_csv(matriz, nombre_archivo): guarda el inventario y su encabezado.
ABM de productos	alta_producto(...): agrega un producto si el código no existe y los datos pasan las validaciones. baja_producto(matriz, codigo): elimina un registro. modificar_stock(matriz, codigo, cantidad): suma o resta unidades sin permitir stock negativo.
Búsquedas	busqueda_secuencial(matriz, codigo): recorre filas de principio a fin. busqueda_binaria(matriz, codigo): divide el rango de búsqueda en mitades, sobre una matriz previamente ordenada por código. Ambas devuelven posición y número de comparaciones.
Ordenamientos	ordenar_seleccion(matriz, columna) y ordenar_burbuja(matriz, columna): ordenan de menor a mayor según la columna elegida, utilizando comparaciones e intercambios programados manualmente.
Reportes	stock_bajo(matriz, umbral): devuelve los productos por debajo del límite. valor_total(matriz): suma precio × stock. producto_mas_caro(matriz) y producto_mas_barato(matriz): recorren la matriz para localizar los precios extremos.
Interacción	menu(): muestra las opciones hasta que se selecciona salir. ventana.py reutiliza las funciones de inventario.py y agrega una visualización animada de los algoritmos de ordenamiento.

3. Comparación y elección de métodos de búsqueda

La búsqueda secuencial revisa los productos uno por uno hasta encontrar el código o terminar la lista. No necesita ordenar los datos y, en el peor caso, realiza tantas comparaciones como productos haya (orden O(n)).

La búsqueda binaria compara el código buscado con el elemento central y descarta la mitad que no corresponde. Requiere que la matriz esté ordenada por código; una vez ordenada, necesita muchas menos comparaciones cuando aumenta la cantidad de registros (orden O(log n)).

Elección para este caso: para una búsqueda aislada en el inventario pequeño y desordenado conviene la secuencial, porque evita ordenar antes. Si la lista ya está ordenada por código y se hacen muchas consultas, conviene la binaria. En este programa, antes de la búsqueda binaria se aplica selección por código; ese costo previo también debe considerarse.

4. Comparación y elección de métodos de ordenamiento

Selección busca el elemento menor entre los que faltan ordenar y lo ubica en la posición correspondiente. Burbuja compara elementos vecinos e intercambia los que están fuera de orden. Los dos algoritmos trabajan sobre la matriz y admiten ordenar por nombre, precio o stock.

Rendimiento: con la implementación utilizada, ambos realizan $n(n-1)/2$ comparaciones, por lo que su costo de comparación es $O(n^2)$. Selección generalmente necesita menos intercambios de filas que burbuja; burbuja es especialmente fácil de seguir visualmente.

Elección para este caso: para ordenar los 15 productos, selección es una opción conveniente por su menor cantidad de intercambios. Burbuja también es válido y resulta útil en la interfaz para mostrar paso a paso cómo se comparan e intercambian productos. Ninguno es ideal para inventarios muy grandes.

5. Reparto del trabajo entre integrantes

El desarrollo lo hicimos entre los dos, con una participación pareja: 50 % cada uno.

En lugar de dividirnos el proyecto en partes separadas, fuimos trabajando juntos en las distintas funciones del sistema. A medida que avanzábamos, íbamos probando lo que hacíamos y ajustando lo que fuera necesario.

Cuando alguno tenía una duda o se trababa con una parte, lo veíamos entre los dos y buscábamos la forma de resolverlo. Así nos fuimos complementando durante todo el desarrollo.

6. Declaración del uso de inteligencia artificial

Para la interfaz gráfica opcional de tkinter se utilizó Claude como apoyo, tal como se declara en ventana.py. ChatGPT se utilizó para revisar el proyecto frente a la consigna y asistir en la elaboración de un borrador del informe. En la presentación final se utilizó especialmente para estandarizar la estética, la maquetación y el formato del documento. El contenido técnico se basa en la consigna y el código presentado.

Archivos del proyecto: inventario.py · ventana.py · inventario.csv · inventario.json